FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
Curso de Ciência da Computação
6º Semestre — Turma 6º CCOMP

SWING SENSE

TECNOLOGIAS E FERRAMENTAS DO PROTÓTIPO — ENTREGA 2

(Sistemas Embarcados e Robótica)

Grupo 1

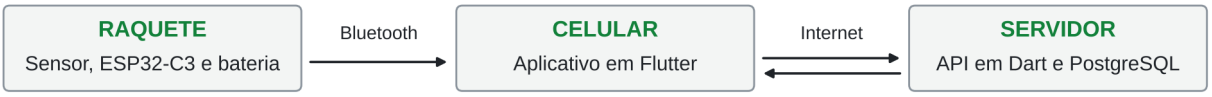
Antônio Petri de Moraes Soares de Moura e Oliveira
Gabriel Higobassi Paschoal
Lucas Silva Maciel
Thiago Akira Higa Mitami

São Paulo
2026

1 O QUE É O PROJETO

O Swing Sense é um módulo eletrônico preso ao cabo da raquete de tênis que percebe cada golpe e mostra os resultados no celular, como um relógio esportivo faz com uma corrida: o jogador treina normalmente e depois vê quantos golpes deu, de que tipo e com que intensidade. É um projeto de Internet das Coisas (IoT): um objeto físico que coleta dados e os envia a programas que os guardam e apresentam.

Figura 1 – Caminho do dado, da raquete ao servidor



Fonte: elaborado pelos autores (2026).

2 COMO FUNCIONA

Medir. O sensor MPU6050 mede quanto a raquete acelera e gira (INVENSENSE, 2013). Cerca de cem vezes por segundo, esses números são lidos pelo ESP32-C3, um computador inteiro em um só chip (ESPRESSIF SYSTEMS, 2021), que roda um programa chamado firmware (Apêndice A).

Reconhecer. O firmware segue o gesto em quatro fases (máquina de estados): parado; preparação, quando a raquete começa a girar; impacto, quando a aceleração passa de um limite; e recuperação, até o movimento acabar. No impacto, a aceleração em um eixo do sensor indica o golpe: *forehand*, com a palma da mão à frente, ou *backhand*, com as costas da mão. Ao fim, o módulo calcula os picos de aceleração e giro e estima a velocidade da raquete.

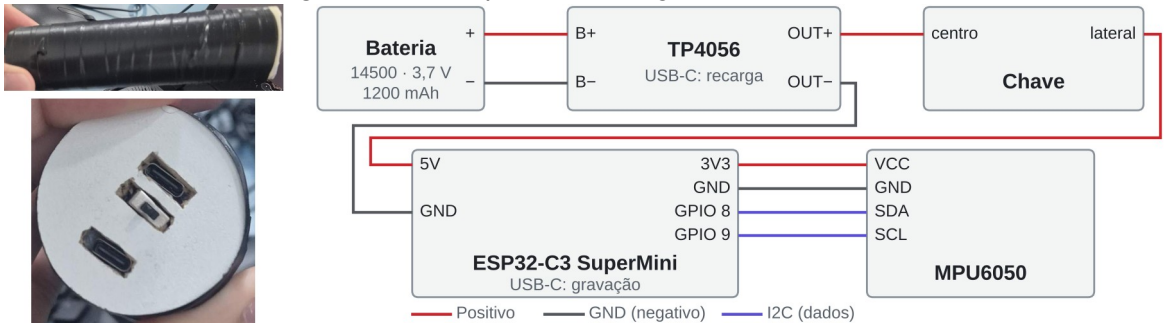
Enviar. Cada golpe vira uma mensagem curta de texto (JSON), enviada por Bluetooth de baixo consumo (BLE). A raquete não acessa a internet: o celular é o intermediário. O aplicativo mostra o golpe na hora e o guarda em uma fila, o que evita perdas se a internet da quadra cair.

Guardar e mostrar. Da fila, os golpes seguem para o servidor, onde uma API (programa que atende pedidos de outros programas) confere os dados e os grava no banco de dados. Com esse histórico, o aplicativo monta painéis com gráficos e metas.

3 MONTAGEM DO PROTÓTIPO

O circuito foi soldado e acomodado em um tubo plástico encapado com fita (Figura 2). A energia sai da bateria, passa pelo TP4056, que cuida da recarga, e pela chave, e chega ao pino 5V do ESP32-C3, que alimenta o sensor com 3,3 V. Sensor e chip trocam dados por dois fios (protocolo I2C).

Figura 2 – Protótipo montado e guia de solda do circuito



Fonte: elaborado pelos autores (2026).

4 TECNOLOGIAS E FERRAMENTAS

Quadro 1 – Tecnologias e ferramentas por parte do sistema

Parte	Tecnologia	O que é e para que serve
Raquete	ESP32-C3 SuperMini	Microcontrolador com Bluetooth; executa o firmware
	MPU6050 (GY-521)	Sensor de aceleração e de giro em três eixos
	Bateria, TP4056 e chave	Bateria de lítio, circuito de recarga e liga/desliga
Firmware	C/C++ e Arduino IDE	Linguagem do firmware e ferramenta que o grava
	Bibliotecas Adafruit e BLE	Código pronto para ler o sensor e usar o Bluetooth
Comunicação	I2C, BLE e JSON	Ligação do sensor, Bluetooth e formato da mensagem
Aplicativo	Flutter e Dart	Um só código para o aplicativo em Android e iOS
	flutter_blue_plus, fl_chart	Bibliotecas de conexão Bluetooth e de gráficos
Servidor	Dart com shelf	API que recebe os golpes e entrega os painéis
	PostgreSQL (Supabase)	Banco de dados na nuvem: usuários, sessões e golpes
	JWT e bcrypt	Login: credencial temporária e senha protegida
Apoio	Wokwi e multímetro	Simulação do circuito e medição de tensão e corrente
	nRF Connect	Aplicativo para testar as mensagens Bluetooth
	VS Code, Git e GitHub	Edição do código, versões e trabalho em equipe
	GitHub Actions e Docker	Testes automáticos e empacotamento do servidor

Fonte: elaborado pelos autores, com base em Oliveira *et al.* (2026).

5 SITUAÇÃO ATUAL E LIMITAÇÕES

O caminho completo funciona, da raquete ao banco de dados. O saque saiu desta versão, pois os dados não permitiram separá-lo do *forehand*. Nos 13 golpes gravados, os 10 *forehands* e *backhands* foram classificados corretamente, mas o teste usou um só jogador e os dados da própria calibração, então não mede a precisão real. O módulo ainda não detecta se a bola foi atingida nem guarda golpes dados sem o celular conectado.

REFERÊNCIAS

ESPRESSIF SYSTEMS. **ESP32-C3 series datasheet**: version 1.0. Shanghai: Espressif Systems, 2021.

INVENSENSE. **MPU-6000 and MPU-6050 product specification**: revision 3.4. Sunnyvale: InvenSense, 2013.

OLIVEIRA, A. P. M. S. M. *et al*. **Swing Sense**. São Paulo: FECAP, 2026. Disponível em: https://github.com/2026-2-NCC6/Projeto1. Acesso em: 3 nov. 2026.

APÊNDICE A – CÓDIGO-FONTE DO FIRMWARE DO PROTÓTIPO V3

Como complemento, este apêndice reproduz na íntegra os dois arquivos do firmware da versão V3, disponíveis na pasta SRC/Circuito/Prototipo_v3 do repositório do grupo (OLIVEIRA *et al.*, 2026).

A.1 Arquivo passo3_ble_tacada.ino

```
/*
  PASSO 3 — v5 + envio das tacadas por BLE

  ============================================================
  O QUE MUDOU EM RELACAO A v5
  ============================================================
  Deteccao, classificacao, limiares e ranges do sensor sao IGUAIS a v5.
  A unica novidade e que cada tacada, alem de impressa no Serial, e
  enviada por BLE como JSON para o app (contrato: docs/CONTRATO_BLE.md).

  - Nome anunciado: SwingSense-XXXX (XXXX = final do MAC do chip)
  - Modo "ao vivo": sem celular conectado, a tacada so aparece no Serial.
  - Campo "bola" sempre null nesta versao (deteccao de contato = etapa 3).
  - Todo o codigo BLE fica em ble_tacada.h (aba ao lado no Arduino IDE).
  - setup() espera o Monitor Serial por no maximo 8 s (a v5 esperava
    para sempre, o que travaria o prototipo rodando na bateria).
  - A cada 5 s imprime "[vivo] ..." com o estado e se o BLE esta conectado.

  Nao precisa instalar biblioteca nova: o BLE vem no core ESP32.

  ============================================================
  COMO TESTAR (celular com o app nRF Connect)
  ============================================================
  1. Grave o sketch e abra o Monitor Serial (115200). Deve aparecer
     "BLE: anunciando como SwingSense-XXXX".
  2. No nRF Connect: SCAN -> CONNECT em SwingSense-XXXX.
     No Android, antes de ativar as notificacoes, peca MTU 185
     (menu de 3 pontos -> Request MTU).
  3. Abra o servico 5f650a50-... e toque no icone de setas
     (notificacoes) da caracteristica efae8d7d-...
  4. Faca golpes: cada um aparece como texto JSON no celular, igual a
     linha "json:" do Monitor Serial.
*/

#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
#include "ble_tacada.h"

Adafruit_MPU6050 mpu;

enum EstadoTacada { PARADO, PREPARACAO, IMPACTO, RECUPERACAO };
EstadoTacada estadoAtual = PARADO;

// ---------- Limiares de transicao de estado ----------
const float LIMIAR_GIRO_PREPARACAO          = 1.5;   // rad/s
const float LIMIAR_ACEL_IMPACTO            = 20.0;  // m/s2
const float LIMIAR_ACEL_ESTAVEL            = 13.5;  // m/s2
const unsigned long TIMEOUT_PREPARACAO       = 1000; // ms
const unsigned long TEMPO_ESTAVEL_RECUPERACAO = 150;  // ms
const unsigned long TIMEOUT_RECUPERACAO_SEG   = 2000; // ms

// ---------- Classificacao ----------
const float LIMIAR_ACCX_BACKHAND = 14.0;
const float ZONA_INCERTA         = 4.0;

// ---------- Forca / velocidade ----------
```

```
const float RAIO_EFETIVO_M = 0.6;  // pulso -> centro das cordas (ajuste se quiser)

// ---------- Buffer circular (classificacao) ----------
const int TAM_BUFFER      = 16;
const int JANELA_AMOSTRAS = 5;

float bufferAccX[TAM_BUFFER];
int   posBuffer = 0;
int   amostrasNoBuffer = 0;

unsigned long marcaTempo = 0;
unsigned long tempoEstavelDesde = 0;

// Dados da tacada em andamento
String tipoAtual = "";
float  accX_janelaAtual = 0;
float  picoAccelMag = 0;
float  picoGiroMag  = 0;
unsigned long tempoImpacto = 0;

int contForehand = 0;
int contBackhand = 0;

// ---------- Identificacao das tacadas (contrato BLE) ----------
uint32_t idBoot = 0;       // sorteado a cada vez que o ESP32 liga
uint32_t seqTacada = 0;    // 1, 2, 3... desde que ligou

void setup() {
  Serial.begin(115200);
  // espera o Monitor Serial no maximo 8 s: na bateria (sem USB) o
  // while(!Serial) da v5 travaria aqui para sempre e o BLE nunca iniciaria.
  // No ESP32-C3 a porta USB reconecta depois do upload/reset, entao 2 s era
  // pouco e as mensagens de inicio se perdiam antes do monitor abrir.
  unsigned long inicioEspera = millis();
  while (!Serial && millis() - inicioEspera < 8000) delay(10);
  delay(500);  // folga pro monitor terminar de conectar

  Serial.println("Iniciando MPU6050...");
  if (!mpu.begin()) {
    Serial.println("ERRO: nao encontrou o MPU6050. Confira os fios!");
    while (1) delay(10);
  }

  mpu.setAccelerometerRange(MPU6050_RANGE_16_G);
  mpu.setGyroRange(MPU6050_RANGE_2000_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

  idBoot = esp_random();

  // nome com o final do MAC, pra distinguir prototipos
  uint64_t mac = ESP.getEfuseMac();
  char nome[20];
  snprintf(nome, sizeof(nome), "SwingSense-%02X%02X",
           (uint8_t)(mac >> 32), (uint8_t)(mac >> 40));
  bleIniciar(nome);
  Serial.print("BLE: anunciando como ");
  Serial.println(nome);

  delay(100);
  Serial.println("Pronto: FOREHAND x BACKHAND + forca pelo pico real + BLE.\n");
}

void guardarNoBuffer(float valor) {
  bufferAccX[posBuffer] = valor;
  posBuffer = (posBuffer + 1) % TAM_BUFFER;
  if (amostrasNoBuffer < TAM_BUFFER) amostrasNoBuffer++;
}
```

```
float mediaUltimas(int n) {
  if (n > amostrasNoBuffer) n = amostrasNoBuffer;
  if (n == 0) return 0;
  float soma = 0;
  for (int i = 1; i <= n; i++) {
    int idx = (posBuffer - i + TAM_BUFFER) % TAM_BUFFER;
    soma += bufferAccX[idx];
  }
  return soma / n;
}

void loop() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  guardarNoBuffer(a.acceleration.x);

  float accelMag = sqrt(a.acceleration.x * a.acceleration.x +
                        a.acceleration.y * a.acceleration.y +
                        a.acceleration.z * a.acceleration.z);
  float giroMag = sqrt(g.gyro.x * g.gyro.x +
                       g.gyro.y * g.gyro.y +
                       g.gyro.z * g.gyro.z);

  unsigned long agora = millis();

  // sinal de vida a cada 5 s (so aparece no Serial; ajuda a ver que esta rodando)
  static unsigned long ultimoStatus = 0;
  if (agora - ultimoStatus >= 5000) {
    ultimoStatus = agora;
    Serial.print("[vivo] estado=");
    Serial.print(estadoAtual);
    Serial.print("  accelMag=");
    Serial.print(accelMag, 1);
    Serial.print("  BLE=");
    Serial.println(bleConectado ? "conectado" : "aguardando celular");
  }

  switch (estadoAtual) {

    case PARADO:
      if (giroMag > LIMIAR_GIRO_PREPARACAO) {
        estadoAtual = PREPARACAO;
        marcaTempo = agora;
        picoAccelMag = 0;
        picoGiroMag  = 0;
      }
      break;

    case PREPARACAO:
      // o giro maximo pode acontecer ainda na aceleracao da raquete
      if (giroMag > picoGiroMag) picoGiroMag = giroMag;

      if (accelMag > LIMIAR_ACEL_IMPACTO) {
        if (accelMag > picoAccelMag) picoAccelMag = accelMag;
        tempoImpacto = agora;
        classificarNoImpacto();
        estadoAtual = IMPACTO;
        marcaTempo = agora;
      } else if (agora - marcaTempo > TIMEOUT_PREPARACAO) {
        estadoAtual = PARADO;
      }
      break;

    case IMPACTO:
      if (accelMag > picoAccelMag) picoAccelMag = accelMag;
      if (giroMag  > picoGiroMag)  picoGiroMag  = giroMag;
      estadoAtual = RECUPERACAO;
```

```
      marcaTempo = agora;
      tempoEstavelDesde = 0;
      break;

    case RECUPERACAO:
      // continua rastreando: o pico real vem ~100 ms depois do cruzamento
      if (accelMag > picoAccelMag) picoAccelMag = accelMag;
      if (giroMag  > picoGiroMag)  picoGiroMag  = giroMag;

      if (accelMag < LIMIAR_ACEL_ESTAVEL) {
        if (tempoEstavelDesde == 0) {
          tempoEstavelDesde = agora;
        } else if (agora - tempoEstavelDesde > TEMPO_ESTAVEL_RECUPERACAO) {
          reportarTacada();
          estadoAtual = PARADO;
        }
      } else {
        tempoEstavelDesde = 0;
      }

      if (agora - marcaTempo > TIMEOUT_RECUPERACAO_SEG) {
        reportarTacada();
        estadoAtual = PARADO;
      }
      break;
  }

  delay(10);  // ~100 Hz
}

// Classificacao continua no impacto (janela pre-impacto do buffer)
void classificarNoImpacto() {
  accX_janelaAtual = mediaUltimas(JANELA_AMOSTRAS);
  if (accX_janelaAtual > LIMIAR_ACCX_BACKHAND) {
    tipoAtual = "BACKHAND";
    contBackhand++;
  } else {
    tipoAtual = "FOREHAND";
    contForehand++;
  }
}

// Forca so e reportada quando o golpe termina, com o pico completo
void reportarTacada() {
  float velRaquete_ms  = picoGiroMag * RAIO_EFETIVO_M;
  float velRaquete_kmh = velRaquete_ms * 3.6;
  float picoG = picoAccelMag / 9.81;
  bool incerto = fabs(accX_janelaAtual - LIMIAR_ACCX_BACKHAND) < ZONA_INCERTA;

  Serial.print(">> ");
  Serial.print(tipoAtual);
  Serial.print("   accX_janela=");
  Serial.print(accX_janelaAtual, 2);
  if (incerto) {
    Serial.print(" (perto do limiar)");
  }
  Serial.println();

  Serial.print("   pico acel: ");
  Serial.print(picoAccelMag, 1);
  Serial.print(" m/s2 (");
  Serial.print(picoG, 1);
  Serial.print(" g)   pico giro: ");
  Serial.print(picoGiroMag, 1);
  Serial.print(" rad/s   vel. estimada: ~");
  Serial.print(velRaquete_kmh, 0);
  Serial.println(" km/h");
```

```
  Serial.print("   total -> forehand: ");
  Serial.print(contForehand);
  Serial.print(" | backhand: ");
  Serial.println(contBackhand);

  // ---------- JSON do contrato BLE (docs/CONTRATO_BLE.md) ----------
  seqTacada++;
  char json[200];
  snprintf(json, sizeof(json),
           "{\"v\":1,\"boot\":%lu,\"seq\":%lu,\"t\":%lu,\"tipo\":\"%s\","
           "\"incerto\":%s,\"bola\":null,\"acel\":%.1f,\"giro\":%.1f,"
           "\"vel\":%d,\"accx\":%.1f}",
           (unsigned long)idBoot, (unsigned long)seqTacada,
           (unsigned long)tempoImpacto,
           tipoAtual == "BACKHAND" ? "BH" : "FH",
           incerto ? "true" : "false",
           picoAccelMag, picoGiroMag,
           (int)(velRaquete_kmh + 0.5), accX_janelaAtual);

  Serial.print("   json: ");
  Serial.println(json);
  Serial.println();

  bleEnviarTacada(json);
}
```

A.2 Arquivo ble_tacada.h

```
/*
  ble_tacada.h — envio das tacadas por BLE (contrato: docs/CONTRATO_BLE.md)

  Usa a biblioteca BLE que ja vem no core ESP32 (3.x); nao precisa instalar nada.
  O ESP32-C3 so tem BLE (nao tem Bluetooth Classico / BluetoothSerial).

  Interface usada pelo .ino:
    bleIniciar(nome)        -> cria servico/caracteristica e comeca a anunciar
    bleEnviarTacada(json)   -> manda uma tacada, se houver celular conectado
*/

#pragma once

#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLE2902.h>

#define UUID_SERVICO_SWINGSENSE "5f650a50-3eba-436c-b9d7-5beb09160c55"
#define UUID_CARAC_TACADA       "efae8d7d-880d-40d4-9a82-bc63cf7a3d81"

static BLEServer*         bleServidor    = nullptr;
static BLECharacteristic* bleCaracTacada = nullptr;
static volatile bool      bleConectado   = false;

class SwingSenseCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer* servidor) override {
    bleConectado = true;
    Serial.println("BLE: celular conectado");
  }

  void onDisconnect(BLEServer* servidor) override {
    bleConectado = false;
    Serial.println("BLE: celular desconectado, anunciando de novo");
    servidor->startAdvertising();
  }
};

void bleIniciar(const char* nome) {
  BLEDevice::init(nome);
```

```
  bleServidor = BLEDevice::createServer();
  bleServidor->setCallbacks(new SwingSenseCallbacks());

  BLEService* servico = bleServidor->createService(UUID_SERVICO_SWINGSENSE);
  bleCaracTacada = servico->createCharacteristic(
    UUID_CARAC_TACADA, BLECharacteristic::PROPERTY_NOTIFY);

#if defined(CONFIG_BLUEDROID_ENABLED)
  // descritor 0x2902 (liga/desliga notificacoes); o NimBLE cria sozinho
  bleCaracTacada->addDescriptor(new BLE2902());
#endif

  servico->start();

  BLEAdvertising* anuncio = BLEDevice::getAdvertising();
  anuncio->addServiceUUID(UUID_SERVICO_SWINGSENSE);
  // UUID de 128 bits + nome nao cabem juntos no anuncio (31 bytes):
  // o nome vai na resposta de scan
  anuncio->setScanResponse(true);
  BLEDevice::startAdvertising();
}

void bleEnviarTacada(const char* json) {
  if (!bleConectado || bleServidor == nullptr) return;

  // payload de uma notificacao = MTU - 3 bytes de cabecalho ATT
  size_t tamanho = strlen(json);
  uint16_t mtu = bleServidor->getPeerMTU(bleServidor->getConnId());
  if (tamanho > (size_t)(mtu - 3)) {
    Serial.print("AVISO BLE: MTU pequeno (");
    Serial.print(mtu);
    Serial.print(") para JSON de ");
    Serial.print(tamanho);
    Serial.println(" bytes. O app deve pedir MTU >= 185. Tacada nao enviada.");
    return;
  }

  // so chega ao celular se ele ativou as notificacoes da caracteristica
  bleCaracTacada->setValue((uint8_t*)json, tamanho);
  bleCaracTacada->notify();
}
```